Molfig

Native molecular rendering for Typst

v0.2.0 2026-09-01 MIT

Render molecular structures with a Mol*-oriented native renderer.

Eito Yoneyama

Molfig renders PDB, mmCIF, BinaryCIF, and XYZ molecular structures in static Typst documents. Version 0.2.0 replaces the former mesh-renderer bridge with a deterministic Rust/WASM renderer designed around the pinned Mol* camera, material, lighting, and Quick Style state.

Table of Contents

Part I

Overview

Molfig renders PDB, mmCIF, BinaryCIF, and XYZ molecular structures in static Typst documents. Version 0.2.0 replaces the former mesh-renderer bridge with a deterministic Rust/WASM renderer designed around the pinned Mol* camera, material, lighting, and Quick Style state. The normal rendering path returns RGBA8 pixels directly and does not serialize OBJ, STL, or PLY.

Part II

Installation

```
#import "@preview/molfig:0.2.0"
```

The `data` argument accepts bytes, an inline string, or a Typst 0.15+ path. Use `read("structure.pdb", encoding: none)` when the document must also work with Typst 0.14.

Part III

First render

```
// Structural data: RCSB PDB / wwPDB entry 9R1O (CC0 1.0).
#let pdb = read("9R1O.pdb", encoding: none)

#molfig.render(
  pdb,
  format: "pdb",
  representation: "cartoon",
  assembly: "1",
  quality: "high",
  renderer: (
    viewport: (width: 1200, height: 900, pixel-ratio: 1),
    camera: (
      view: (name: "orbit", params: (azimuth: 35, elevation: 24)),
    ),
    background: (color: "#ffffff", transparent: true),
  ),
)
```

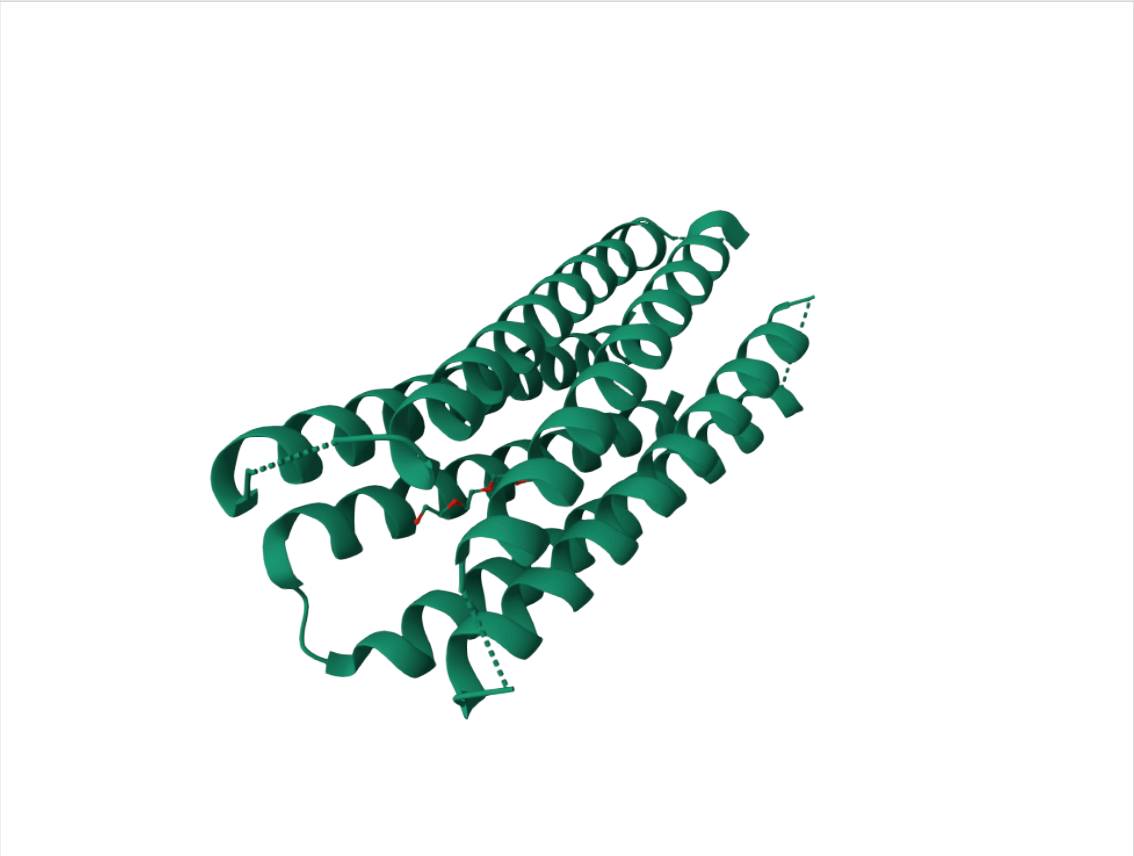

Figure 1: RCSB PDB entry 9R1O rendered as Cartoon.

`renderer.viewport` controls deterministic pixel dimensions. `width` and `height` on `render` control only the size of the image in Typst layout.

Part IV
Rendering API

IV.1 `render`

`render(data, ..., renderer: (:), output-format: "svg", width: auto, height: auto)` returns Typst image content backed by native RGBA8 pixels. `output-format` accepts `"svg"` and `"png"`; the default is `"svg"`. Since Mol* SSAO, depth outlines, transparency, and antialiasing are screen-space operations, SVG output embeds the lossless PNG raster instead of converting the molecule to vector geometry.

The rendering-specific arguments removed in 0.2.0 are `mesh-format`, `config`, render-time `center`, and render-time `style-params`.

IV.2 `render-result`

`render-result` accepts the same arguments and returns:

- `content`: ready-to-place Typst image content;
- `image`: encoded SVG or PNG bytes;
- `output-format`: the selected image format;
- `pixels`: raw RGBA8 bytes;
- `pixel-width` and `pixel-height`;
- `info`: molecular and realized-representation metadata; its `render_objects` field is the compact list of actual grouped render objects and equals `render-info.render_objects`;
- `render-info`: resolved renderer, camera, style, and pass metadata.

```
#let result = molfig.render-result(
  read("1CRN.bcif", encoding: none),
  format: "bcif",
  representation: "cartoon",
  renderer: (viewport: (width: 960, height: 720)),
)

#result.content
Atoms: #result.info.atom_count\
Pixels: #result.pixel-width × #result.pixel-height
```

`render-object` and `mesh-info` were removed. Use `render-result` for rendered pixels and metadata, `info` for metadata without rasterization, or an explicit export function for interchange geometry.

Part V

Renderer dictionary

The renderer dictionary is decoded by a strict typed schema. Unknown keys, unsupported mapped values, invalid colors, and non-finite or out-of-range numbers are errors.

viewport default: `(width: 800, height: 800, pixel-ratio: 1)`
Pixel dimensions before and after the pixel-ratio multiplier. Width and height are integers from 1 through 4096; pixel ratio is 0.25 through 4.

camera default: `(mode: "perspective", fov: 45, ...)`
Projection, view, fog, and clipping. `mode` is `"perspective"` or `"orthographic"`. `fov` is measured in degrees. Clipping accepts Mol* snapshot values `far`, `min-near`, and `min-far`.

background default: `(color: "#fcfbfa", transparent: false)`
A six-digit hexadecimal color and alpha policy.

shading default: `(ignore-light: auto, material: (...))`
An explicit Boolean `ignore-light` overrides the style preset. Mol* Matte material defaults are metalness 0, roughness 1, and bumpiness 0. Bumpiness must remain 0 until representation-specific bump frequency is supported.

lighting default: `(exposure: 1, ambient: ..., directional: ...)`
Mol* defaults are white ambient intensity 0.4 and one white directional light at inclination 150°, azimuth 320°, intensity 0.6.

transparency default: `(mode: "wboit")`
Names the Mol*-compatible transparency path. Other names are rejected.

multi-sample default: `(mode: "temporal", sample-level: 2, reuse-occlusion: true)`
`mode` is `"off"`, synchronous `"on"`, or `"temporal"`. Sample level is an integer from 0 through 5. `reuse-occlusion` reuses the first SSAO result across jittered samples, matching the corresponding Mol* option.

postprocessing default: `(occlusion: auto, outline: auto, shadow: auto, antialiasing: auto)`
Occlusion accepts `name` plus `params` containing `samples`, `multi-scale`, `radius`, `bias`, `blur-kernel-size`, `blur-depth-bias`, `resolution-scale`, `color`, and `transparent-threshold`. `multi-scale` is a mapped `on`/`off` value; its `on` parameters are `levels` (radius and bias pairs), `near-threshold`, and `far-threshold`. The single `radius` is used when multi-scale is off. Resolution scale is combined with pixel ratio using Mol*'s SSAO target-size rule. Outline accepts `name` plus `color`, `scale`, `threshold`, and `include-transparent` parameters. Shadow accepts only `name` and must remain off. Antialiasing is `"smaa"` with `edge-threshold` and `max-search-steps` parameters, or `"off"`.

V.1 Camera views

`camera.view` is a mapped dictionary.

- `auto` fits the visible bounding sphere with the initial +Z view direction and +Y up.
- `orbit` applies authoring-friendly `azimuth`, `elevation`, and `roll` to the same fitted target and distance.
- `snapshot` accepts explicit `position`, `target`, `up`, `radius`, and `radius-max` and is the strict interchange form.

View parameters are strict and variant-specific: `auto` takes none, `orbit` accepts only its three angle parameters, and `snapshot` requires all five snapshot parameters. Parameters belonging to another view are errors.

```
#molfig.render(
  data,
  renderer: (
    viewport: (width: 1200, height: 900, pixel-ratio: 1),
    camera: (
      mode: "perspective",
      fov: 45,
      view: (
        name: "orbit",
        params: (azimuth: 35, elevation: 24, roll: 0),
      ),
      fog: (name: "on", params: (intensity: 15)),
      clipping: (far: true, min-near: 1, min-far: 0),
    ),
    background: (color: "#ffffff", transparent: true),
  ),
)
```

Part VI

Input formats

- `format: "pdb"` reads PDB text.
- `format: "cif"` or `"mmcif"` reads text mmCIF.
- `format: "bcif"` reads BinaryCIF.
- `format: "xyz"` reads one or more XYZ frames. Missing bonds are inferred with Mol*-compatible element and distance rules unless `infer-bonds` is false.
- `format: "auto"` detects supported formats, but explicit formats are more reproducible.

For CIF and BinaryCIF, use `block-index` or `block-header` to choose a data block. Use `assembly` for a biological assembly or `"asymmetric-unit"`, and `alt-loc` for alternate-location selection.

Part VII

XYZ example

```
// PubChem CID 702, PubChem3D conformer 000002BE00000001.
#let xyz = read("ethanol.xyz", encoding: none)

#molfig.render(
  xyz,
  format: "xyz",
  representation: "default",
  color-theme: "element-symbol",
  quality: "high",
  renderer: (
    camera: (
      view: (name: "orbit", params: (azimuth: 125, elevation: 40)),
    ),
    background: (color: "#ffffff", transparent: true),
  ),
  width: 74mm,
)
```

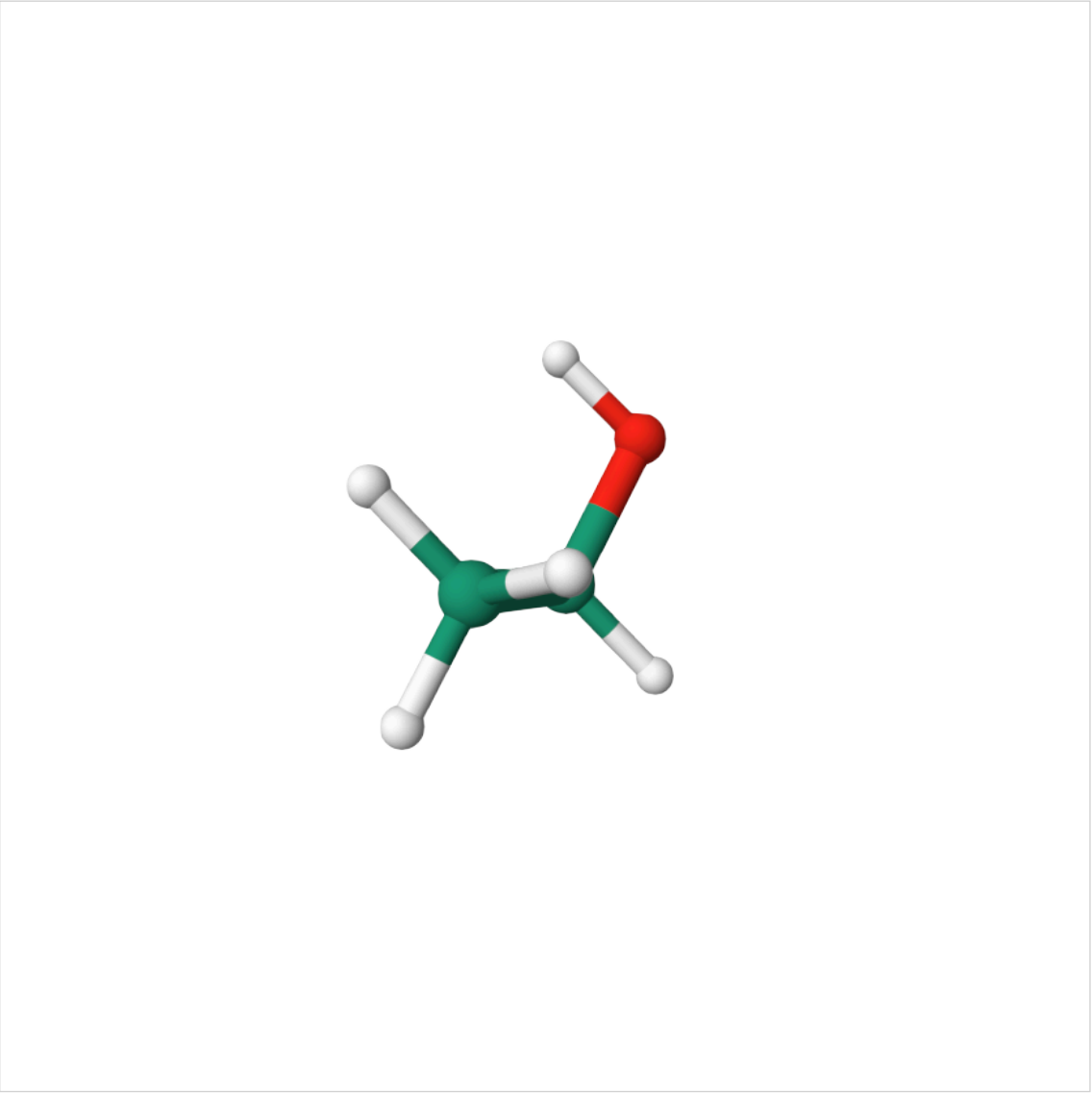

Figure 2: PubChem ethanol rendered from XYZ coordinates.

Coordinate source: PubChem CID 702, PubChem3D conformer `000002BE00000001`, retrieved through PubChem PUG REST. The complete coordinate file and attribution are distributed with the examples.

VII.1 XYZ representations

Small XYZ input resolves `representation: "default"` to the Mol* Viewer ball-and-stick preset. `spacefill` and `surface` can be selected explicitly.

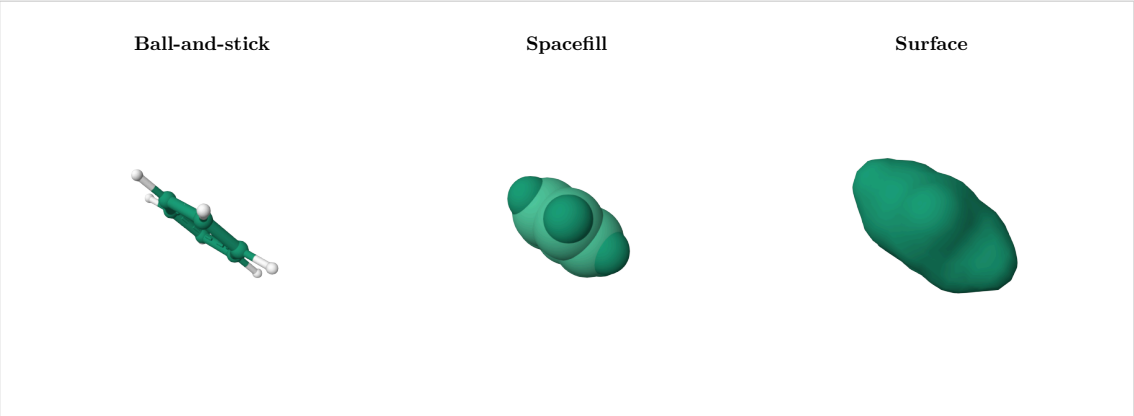

Figure 3: Ball-and-stick, Spacefill, and Surface for the same benzene XYZ conformer.

Part VIII

Representations and color

Supported representation names include `default`, `cartoon`, `polymer-cartoon`, `spacefill`, `ball-and-stick`, `surface`, `ribbon`, and `backbone`. `quality` selects Mol*-oriented geometry detail; `decimate` is a Molfig semantic level-of-detail control from 0 through 1.

Color themes include `chain-id`, `element-symbol`, `entity-id`, `operator-name`, `plddt-confidence`, `qmean-score`, and `sb-ncbr-partial-charges`. `theme` exposes the Viewer-oriented `globalName`, `carbonColor`, and `symmetryColor` overrides.

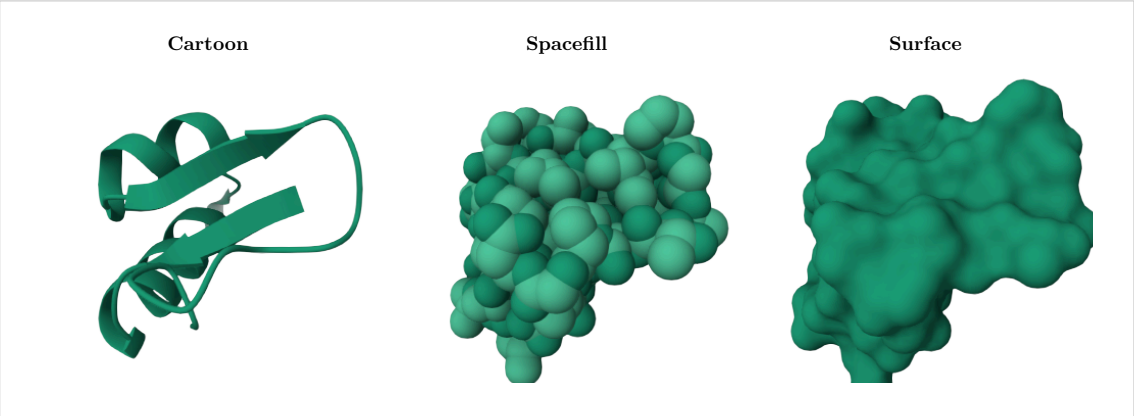

Figure 4: Cartoon, Spacefill, and Surface for RCSB PDB entry 1CRN.

Part IX

Illustrative style

`style: "illustrative"` is orthogonal to representation and color theme. It resolves the Mol* Quick Style intent as unlit material color, SSAO on, black outline on, shadow off, and antialiasing on. Explicit values in `renderer` override the preset.

```
#molfig.render(
  read("1CRN.bcif", encoding: none),
  format: "bcif",
  representation: "cartoon",
  color-theme: "chain-id",
  style: "illustrative",
  output-format: "png",
  renderer: (
    postprocessing: (
      occlusion: (name: "on"),
      outline: (
        name: "on",
        params: (scale: 1, threshold: 0.33, color: "#000000"),
      ),
    ),
  ),
)
```

Figure 5: RCSB PDB entry 1CRN with Illustrative style.

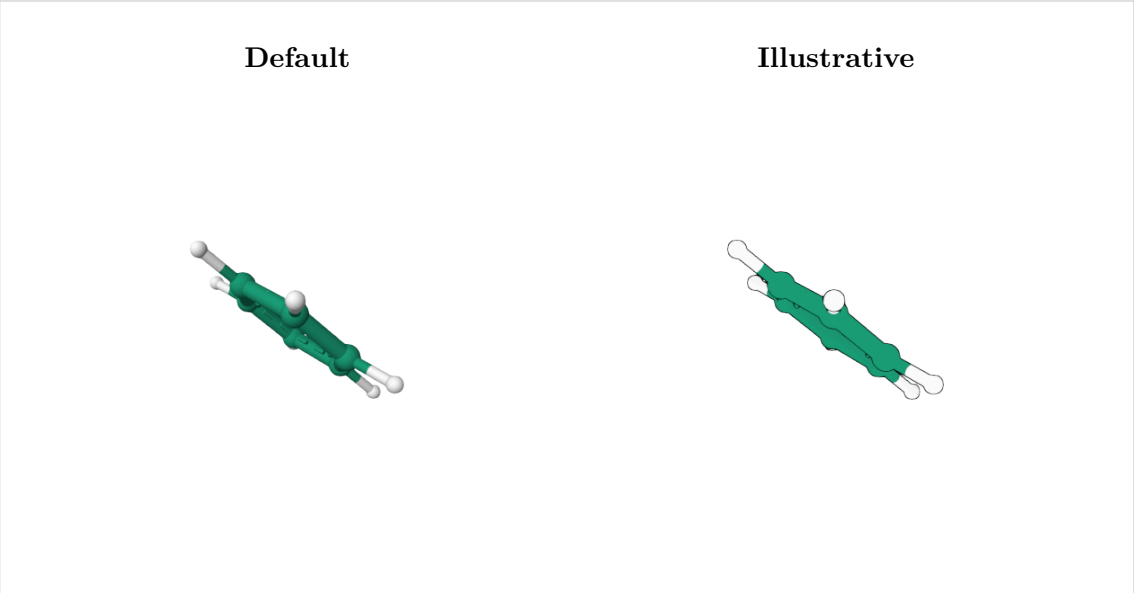

Figure 6: Default and Illustrative styles applied to benzene XYZ.

Part X

Geometry exports

Rendering is independent of export formats.

- `to-obj` returns Wavefront OBJ and preserves groups and material references.
- `to-mtl` returns the companion Mol*-oriented material library.
- `to-stl` returns binary STL geometry.
- `to-ply` remains callable in the 0.2 series as a legacy ASCII exporter. It does not preserve the themed material contract and receives no renderer-specific development.

Export functions retain `center` because coordinate translation is meaningful for interchange files. Native rendering instead fits the camera to the visible scene and has no render-time centering option.

Part XI

Metadata

`info(data, ...)` performs parsing and semantic representation construction without rasterization. It reports atom and bond counts, source data, assemblies, alternate locations, Model/Structure/Unit state, realized visuals, semantic render objects, and bounds.

Part XII

0.2.0 migration

Version 0.2.0 intentionally breaks the former rendering API:

- replace `config` with the typed `renderer` dictionary;
- remove `mesh-format` from rendering calls;
- restrict `output-format` to `"svg"` or `"png"` and change its default to `"svg"`;
- replace `style-params` with `renderer.shading` and `renderer.postprocessing` overrides;
- replace `render-object` with `render-result`;
- remove `mesh-info`;
- do not pass `center` to `render` or `render-result`.

Export APIs remain separate and continue to accept export-specific options.

Part XIII

Reproducibility and limits

Molfig pins the Mol* source revision used as its behavioral reference and reports it through `render-info.molstar_commit`. For reproducible figures, specify input format, representation, assembly, alternate-location policy, quality, viewport, camera view, and background explicitly.

The renderer is CPU based, so high-resolution large assemblies and dense surfaces cost more compilation time and memory than small atomistic or Cartoon figures. Reduce viewport dimensions, geometry quality, or semantic `decimate` when drafting.

Part XIV

License and data attribution

Molfig project code is MIT licensed. Mol* is MIT licensed, copyright Mol* contributors. See `NOTICE.md` and `THIRD_PARTY_NOTICES.md` in the package.

Bundled examples include CC0 PDB archive data from RCSB PDB / wwPDB and PubChem-generated conformers. Per-file identifiers, records, terms, and attributions are listed in `examples/data/README.md`.

Part XV
Index